Code source C++ complet : `main.cpp`

Compilation : `g++ -std=c++17 -Wall -o agence main.cpp`

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <memory>
#include <stdexcept>
#include <algorithm>
using namespace std;

// ---------- Utilitaires de saisie ----------
string lireLigne(const string& msg) {
    cout << msg;
    string s;
    if (!getline(cin, s)) throw runtime_error("Fin de saisie.");
    return s;
}
int lireEntier(const string& msg) {
    while (true) {
        string s = lireLigne(msg);
        try {
            size_t pos; int v = stoi(s, &pos);
            if (pos != s.size()) throw invalid_argument("");
            return v;
        } catch (...) { cout << "  Saisie invalide : entier attendu.\n"; }
    }
}
double lireReel(const string& msg) {
    while (true) {
        string s = lireLigne(msg);
        try {
            size_t pos; double v = stod(s, &pos);
            if (pos != s.size()) throw invalid_argument("");
            return v;
        } catch (...) { cout << "  Saisie invalide : nombre attendu.\n"; }
    }
}
string minuscules(string s) {
    transform(s.begin(), s.end(), s.begin(), ::tolower);
    return s;
}

// ---------- Enumerations ----------
enum class StatutReservation { EN_ATTENTE, VALIDEE, ANNULEE };
enum class StatutRemboursement { AUCUN, A_REMBOURSER };

string texte(StatutReservation s) {
    switch (s) {
        case StatutReservation::EN_ATTENTE: return "EN_ATTENTE";
        case StatutReservation::VALIDEE:    return "VALIDEE";
        default:                            return "ANNULEE";
    }
}
string texte(StatutRemboursement s) {
    return s == StatutRemboursement::AUCUN ? "AUCUN" : "A_REMBOURSER";
}

// ---------- Passager ----------
class Passager {
    int id;
    string nom, prenom, telephone;
public:
    Passager(int id, const string& nom, const string& prenom, const string& tel)
        : id(id), nom(nom), prenom(prenom), telephone(tel) {
        if (nom.empty() || prenom.empty() || tel.empty())
            throw invalid_argument("Nom, prenom et telephone sont obligatoires.");
    }
    int getId() const { return id; }
    string getNom() const { return nom; }
    string getPrenom() const { return prenom; }
    string getTelephone() const { return telephone; }
    void afficher() const {
        cout << "  Passager #" << id << " : " << prenom << " " << nom
             << " (" << telephone << ")\n";
    }
};

```

```
// ---------- Vol ----------
class Vol {
    string numero, depart, destination, date;
    int capacite, placesReservees;
    double prix;
public:
    Vol(const string& numero, const string& depart, const string& dest,
        const string& date, int capacite, double prix)
        : numero(numero), depart(depart), destination(dest), date(date),
          capacite(capacite), placesReservees(0), prix(prix) {
        if (numero.empty() || depart.empty() || dest.empty() || date.empty())
            throw invalid_argument("Tous les champs du vol sont obligatoires.");
        if (capacite <= 0) throw invalid_argument("La capacite doit etre positive.");
        if (prix <= 0) throw invalid_argument("Le prix doit etre positif.");
    }
    string getNumero() const { return numero; }
    string getDepart() const { return depart; }
    string getDestination() const { return destination; }
    double getPrix() const { return prix; }
    int placesDisponibles() const { return capacite - placesReservees; }
    void bloquerPlace() {
        if (placesDisponibles() <= 0)
            throw runtime_error("Vol complet : plus de place disponible.");
        placesReservees++;
    }
    void libererPlace() { if (placesReservees > 0) placesReservees--; }
    void afficher() const {
        cout << "  Vol " << numero << " : " << depart << " -> " << destination
             << " | " << date << " | " << prix << " FCFA | places libres : "
             << placesDisponibles() << "/" << capacite << "\n";
    }
};

// ---------- Paiement ----------
class Paiement {
    int id;
    double montant;
    string mode;
public:
    Paiement(int id, double montant, const string& mode)
        : id(id), montant(montant), mode(mode) {}
    int getId() const { return id; }
    double getMontant() const { return montant; }
    string getMode() const { return mode; }
};

// ---------- Billet ----------
class Billet {
    string numero;
    Passager* passager;
    Vol* vol;
    bool valide;
public:
    Billet(const string& numero, Passager* p, Vol* v)
        : numero(numero), passager(p), vol(v), valide(true) {}
    string getNumero() const { return numero; }
    bool estValide() const { return valide; }
    void invalider() { valide = false; }
    void afficher() const {
        cout << "  Billet " << numero << " | " << passager->getPrenom() << " "
             << passager->getNom() << " | vol " << vol->getNumero()
             << " | " << (valide ? "VALIDE" : "INVALIDE") << "\n";
    }
};

// ---------- Reservation ----------
class Reservation {
    int id;
    Passager* passager;
    Vol* vol;
    StatutReservation statut;
    StatutRemboursement remboursement;
    shared_ptr<Paiement> paiement;
    shared_ptr<Billet> billet;
public:
    Reservation(int id, Passager* p, Vol* v)
        : id(id), passager(p), vol(v), statut(StatutReservation::EN_ATTENTE),
          remboursement(StatutRemboursement::AUCUN) {}
    int getId() const { return id; }
    StatutReservation getStatut() const { return statut; }
    Vol* getVol() const { return vol; }
```

```
    Passager* getPassager() const { return passager; }
    bool aPaiement() const { return paiement != nullptr; }

    // Validation = place bloquee definitivement (R5)
    void valider(shared_ptr<Paiement> pay, shared_ptr<Billet> bil) {
        vol->bloquerPlace();            // leve une exception si complet (R2)
        paiement = pay;
        billet = bil;
        statut = StatutReservation::VALIDEE;
    }
    void annuler() {
        if (statut == StatutReservation::ANNULEE)
            throw runtime_error("Reservation deja annulee.");
        if (statut == StatutReservation::VALIDEE) {   // payee -> R6
            vol->libererPlace();
            if (billet) billet->invalider();
            remboursement = StatutRemboursement::A_REMBOURSER;
        }
        statut = StatutReservation::ANNULEE;
    }
    void afficherResume() const {
        cout << "  Reservation #" << id << " | " << passager->getPrenom() << " "
             << passager->getNom() << " | vol " << vol->getNumero()
             << " | " << texte(statut) << "\n";
    }
    void afficher() const {
        cout << "  --- Reservation #" << id << " ---\n";
        cout << "  Passager      : " << passager->getPrenom() << " " << passager->getNom() << "\n";
        cout << "  Vol           : " << vol->getNumero() << " (" << vol->getDepart()
             << " -> " << vol->getDestination() << ")\n";
        cout << "  Statut        : " << texte(statut) << "\n";
        cout << "  Paiement      : ";
        if (paiement) cout << paiement->getMontant() << " FCFA (" << paiement->getMode() << ")\n";
        else cout << "aucun\n";
        cout << "  Billet        : ";
        if (billet) cout << billet->getNumero() << (billet->estValide() ? " (valide)\n" : " (invalide)\n");
        else cout << "non emis\n";
        cout << "  Remboursement : " << texte(remboursement) << "\n";
    }
};

// ---------- Agence ----------
class Agence {
    vector<shared_ptr<Passager>> passagers;
    vector<shared_ptr<Vol>> vols;
    vector<shared_ptr<Reservation>> reservations;
    vector<shared_ptr<Billet>> billets;
    int prochainPassager = 1, prochaineReservation = 1, prochainPaiement = 1;

    shared_ptr<Passager> trouverPassager(int id) const {
        for (auto& p : passagers) if (p->getId() == id) return p;
        throw runtime_error("Passager introuvable (il doit etre enregistre avant).");
    }
    shared_ptr<Vol> trouverVol(const string& num) const {
        for (auto& v : vols) if (minuscules(v->getNumero()) == minuscules(num)) return v;
        throw runtime_error("Vol introuvable.");
    }
    shared_ptr<Reservation> trouverReservation(int id) const {
        for (auto& r : reservations) if (r->getId() == id) return r;
        throw runtime_error("Reservation introuvable.");
    }
public:
    void enregistrerPassager(const string& nom, const string& prenom, const string& tel) {
        for (auto& p : passagers)
            if (p->getTelephone() == tel) throw runtime_error("Un passager avec ce telephone existe deja.");
        passagers.push_back(make_shared<Passager>(prochainPassager, nom, prenom, tel));
        cout << "  Passager enregistre avec l'identifiant " << prochainPassager++ << ".\n";
    }
    void ajouterVol(const string& num, const string& dep, const string& dest,
                    const string& date, int cap, double prix) {
        for (auto& v : vols)
            if (minuscules(v->getNumero()) == minuscules(num)) throw runtime_error("Numero de vol deja utilise.");
        vols.push_back(make_shared<Vol>(num, dep, dest, date, cap, prix));
        cout << "  Vol ajoute.\n";
    }
    void afficherVolsDisponibles() const {
        bool trouve = false;
        for (auto& v : vols)
            if (v->placesDisponibles() > 0) { v->afficher(); trouve = true; }
        if (!trouve) cout << "  Aucun vol disponible.\n";
    }
```

```
    void rechercherVol(const string& dep, const string& dest) const {
        bool trouve = false;
        for (auto& v : vols)
            if (minuscules(v->getDepart()) == minuscules(dep) &&
                minuscules(v->getDestination()) == minuscules(dest)) { v->afficher(); trouve = true; }
        if (!trouve) cout << "  Aucun vol ne correspond a la recherche.\n";
    }
    void reserver(int idPassager, const string& numVol) {
        auto p = trouverPassager(idPassager);
        auto v = trouverVol(numVol);
        if (v->placesDisponibles() <= 0) throw runtime_error("Vol complet : reservation impossible.");
        reservations.push_back(make_shared<Reservation>(prochaineReservation, p.get(), v.get()));
        cout << "  Reservation #" << prochaineReservation++
             << " creee (EN_ATTENTE : la place sera bloquee apres paiement/validation).\n";
    }
    void payer(int idRes, double montant, const string& mode) {
        auto r = trouverReservation(idRes);
        if (r->getStatut() == StatutReservation::ANNULEE)
            throw runtime_error("Reservation annulee : paiement impossible.");
        if (r->getStatut() == StatutReservation::VALIDEE)
            throw runtime_error("Reservation deja payee et validee.");
        if (montant != r->getVol()->getPrix())
            throw runtime_error("Montant incorrect : le prix du billet est de "
                                + to_string((long)r->getVol()->getPrix()) + " FCFA.");
        auto pay = make_shared<Paiement>(prochainPaiement++, montant, mode);
        string num = "BIL-" + to_string(billets.size() + 1);
        auto bil = make_shared<Billet>(num, r->getPassager(), r->getVol());
        r->valider(pay, bil);          // peut lever une exception si le vol est complet
        billets.push_back(bil);
        cout << "  Paiement enregistre. Reservation validee, billet " << num << " emis.\n";
    }
    void annuler(int idRes) {
        auto r = trouverReservation(idRes);
        bool payee = r->aPaiement();
        r->annuler();
        cout << "  Reservation annulee.";
        if (payee) cout << " Remboursement : A_REMBOURSER.";
        cout << "\n";
    }
    void consulterReservations() const {
        if (reservations.empty()) { cout << "  Aucune reservation.\n"; return; }
        for (auto& r : reservations) r->afficherResume();
    }
    void afficherReservation(int id) const { trouverReservation(id)->afficher(); }
};

// ---------- Programme principal ----------
void afficherMenu() {
    cout << "\n===== AGENCE DE VOYAGE - COTONOU =====\n"
         << "1) Enregistrer un passager\n2) Ajouter un vol\n3) Afficher les vols disponibles\n"
         << "4) Rechercher un vol\n5) Effectuer une reservation\n6) Enregistrer un paiement\n"
         << "7) Annuler une reservation\n8) Consulter les reservations\n"
         << "9) Afficher les infos d'une reservation\n10) Quitter\n";
}

int main() {
    Agence agence;
    int choix = 0;
    try {
        while (choix != 10) {
            afficherMenu();
            choix = lireEntier("Votre choix : ");
            try {
                switch (choix) {
                    case 1: {
                        string n = lireLigne("Nom : "), p = lireLigne("Prenom : "), t = lireLigne("Telephone : ");
                        agence.enregistrerPassager(n, p, t); break; }
                    case 2: {
                        string num = lireLigne("Numero du vol : "), d = lireLigne("Depart : "),
                               a = lireLigne("Destination : "), dt = lireLigne("Date (JJ/MM/AAAA) : ");
                        int cap = lireEntier("Capacite : ");
                        double prix = lireReel("Prix (FCFA) : ");
                        agence.ajouterVol(num, d, a, dt, cap, prix); break; }
                    case 3: agence.afficherVolsDisponibles(); break;
                    case 4: {
                        string d = lireLigne("Depart : "), a = lireLigne("Destination : ");
                        agence.rechercherVol(d, a); break; }
                    case 5: {
                        int id = lireEntier("Identifiant du passager : ");
                        string num = lireLigne("Numero du vol : ");
                        agence.reserver(id, num); break; }
```

```
                    case 6: {
                        int id = lireEntier("Numero de reservation : ");
                        double m = lireReel("Montant (FCFA) : ");
                        string mode = lireLigne("Mode (Especes/Mobile Money/Carte) : ");
                        agence.payer(id, m, mode); break; }
                    case 7: agence.annuler(lireEntier("Numero de reservation : ")); break;
                    case 8: agence.consulterReservations(); break;
                    case 9: agence.afficherReservation(lireEntier("Numero de reservation : ")); break;
                    case 10: cout << "Au revoir !\n"; break;
                    default: cout << "  Choix invalide (1 a 10).\n";
                }
            } catch (const exception& e) {
                cout << "  [ERREUR] " << e.what() << "\n";
            }
        }
    } catch (const exception& e) {
        cout << "\n" << e.what() << "\n";
    }
    return 0;
}
```

Fichier README.md

```
# Gestion des vols et billets d'avion - Agence de Cotonou

## Presentation
Application console en C++ (POO) permettant a une agence de voyage de gerer
passagers, vols, reservations, paiements et annulations.

## Fonctionnalites
- Enregistrer un passager (telephone unique)
- Ajouter, afficher et rechercher des vols
- Reserver un vol (passager enregistre obligatoire)
- Enregistrer un paiement : validation de la reservation, blocage de la place, emission du billet
- Annuler une reservation (remboursement A_REMBOURSER si elle etait payee)
- Consulter les reservations et afficher le detail d'une reservation

## Regles de gestion appliquees
Passager enregistre avant reservation ; pas de sur-reservation ; une reservation = un vol ;
paiement refuse pour une reservation annulee ; place bloquee uniquement a la validation ;
reservation payee puis annulee = remboursement a traiter ; un billet = un passager + un vol.

## Concepts POO utilises
Encapsulation, constructeurs avec controle des donnees, accesseurs, associations entre objets
(pointeurs), std::vector comme collections, exceptions pour les erreurs et operations interdites.

## Compilation et execution
    g++ -std=c++17 -Wall -o agence main.cpp
    ./agence            (Windows : agence.exe)

## Structure
- main.cpp        : classes Passager, Vol, Paiement, Billet, Reservation, Agence + menu
- diagramme.puml  : diagramme de classes PlantUML
```